Product Requirements Document (PRD)

Enterprise Document RAG Chatbot

Document Owner	Product/Engineering Lead
Status	Draft v1.0
Last Updated	September 13, 2026
Stakeholders	Engineering, Data/ML, Design, Compliance, End Users

---

1. Executive Summary

Enterprise teams sit on thousands of pages of PDFs — policy manuals, contracts, SOPs, research reports, compliance filings — that are searched manually or not searched at all. Employees waste time hunting for answers buried in documents, and when they do find something, they often can't verify where it came from.

The **Enterprise Document RAG Chatbot** is an internal knowledge assistant that lets employees ask natural-language questions and get accurate, grounded answers pulled directly from company documents — each answer backed by a clickable source and page number, so nothing is taken on faith.

Under the hood it combines semantic search with keyword search (hybrid retrieval), re-ranks results with a cross-encoder for precision, and generates answers through an LLM constrained to the retrieved context. Quality is continuously measured with RAGAS metrics (faithfulness, relevancy, precision, recall) rather than gut feel.

The product goal is simple: **make company knowledge instantly queryable, trustworthy, and auditable** — through an interface that feels like a polished internal tool, not a generic chatbot demo.

---

2. Problem Statement

Pain Point	Impact
Knowledge is locked in unsearchable PDFs	Employees spend 15–30+ min per query manually searching documents
Generic LLM chatbots hallucinate or fabricate answers	Loss of trust; risk in compliance-sensitive contexts
No traceability to source material	Answers can't be verified or cited in decisions/audits
No way to measure answer quality over time	Teams can't tell if the system is actually improving
Off-the-shelf AI tools look and feel like toys	Low adoption in professional/enterprise settings

---

3. Goals & Objectives

3.1 Primary Goals

1. Deliver **grounded, citation-backed answers** from enterprise PDF corpora.
2. Reduce average time-to-answer for document-based questions from minutes to **under 15 seconds**.
3. Achieve **RAGAS faithfulness score ⩾ 0.85** and **context precision ⩾ 0.80** on the evaluation set.
4. Ship a UI that reads as a **professional internal knowledge tool** — not a generic chat widget.

3.2 Non-Goals (Out of Scope for v1)

- Multi-modal input (images, scanned handwriting, video).
- Real-time document editing or write-back into source PDFs.
- Public-facing / external customer deployment.
- Multi-tenant SaaS billing infrastructure.

3.3 Success Metrics (KPIs)

Metric	Target
Faithfulness (RAGAS)	⩾ 0.85
Answer Relevancy (RAGAS)	⩾ 0.85
Context Precision (RAGAS)	⩾ 0.80
Context Recall (RAGAS)	⩾ 0.80
P95 query latency	⩽ 4 seconds
Weekly active users (internal)	⩾ 60% of target department
User-reported "answer was useful" rate	⩾ 90%
Citation click-through rate	⩾ 40% (proxy for trust/verification behavior)

---

4. Target Users & Personas

1. Priya — Compliance Analyst Needs precise, page-cited answers from policy documents for audit trails. Zero tolerance for hallucination.

2. Raj — New Hire in Operations Needs fast answers to "how do we do X" without pinging senior staff. Values speed and simplicity over configurability.

3. Meera — Engineering Manager Wants to query internal architecture docs and RFCs across hundreds of pages without manually skimming.

4. Admin/IT Owner Responsible for uploading/managing document corpora, monitoring system health, and reviewing evaluation dashboards.

---

5. Scope

5.1 In Scope (v1)

- PDF ingestion, parsing, and chunking pipeline
- Hybrid retrieval (dense embeddings + BM25)
- Cross-encoder re-ranking
- Contextual prompt construction with conversational memory
- LLM-based answer generation with inline citations (source + page)
- FastAPI backend with REST endpoints

- Polished web chat interface (non-generic design system)
- RAGAS-based evaluation pipeline
- Basic admin panel for document upload/management

5.2 Phase 2 (Future)

- Role-based access control per document set
- Multi-format ingestion (DOCX, HTML, Confluence, SharePoint)
- Feedback loop (thumbs up/down) feeding into retrieval tuning
- Analytics dashboard for usage and query trends
- SSO integration

---

6. User Stories

ID	As a...	I want to...	So that...
US-01	Employee	Ask a question in plain language	I get an answer without reading full documents
US-02	Employee	See the exact source/page for each answer	I can verify and trust the response
US-03	Employee	Continue a conversation with follow-up questions	I don't have to repeat context
US-04	Compliance Analyst	Export a Q&A session with citations	I can attach it to an audit record
US-05	Admin	Upload new PDF documents	The knowledge base stays current
US-06	Admin	View retrieval/answer quality metrics	I can catch quality regressions early
US-07	Employee	Get a clear "I don't know" instead of a guess	I'm not misled by a fabricated answer

---

7. Functional Requirements

7.1 Document Ingestion Pipeline

- FR-1: System shall accept PDF uploads (single or bulk) via the admin panel.

- FR-2: System shall parse PDF text while preserving page numbers and section structure.
- FR-3: System shall chunk documents using a semantic/recursive strategy (e.g., ~500–800 tokens with overlap) to preserve context continuity.
- FR-4: System shall generate embeddings for each chunk and store them in ChromaDB alongside metadata (document name, page number, section heading, chunk ID).
- FR-5: System shall build/update a BM25 keyword index in parallel with the vector index.

7.2 Retrieval

- FR-6: System shall run hybrid retrieval — dense vector search + BM25 — for every query.
- FR-7: System shall merge and re-rank candidates using a cross-encoder model, returning the top-k most relevant chunks.
- FR-8: System shall support configurable top-k and retrieval weighting (semantic vs. keyword) via backend config.

7.3 Generation

- FR-9: System shall construct a grounded prompt using only retrieved context plus recent conversation history.
- FR-10: System shall generate answers via LLM (OpenAI) constrained to cite sources inline as `[Document Name, p. X]`.
- FR-11: System shall return "I don't have enough information in the provided documents" when retrieval confidence is low, rather than fabricating an answer.
- FR-12: System shall maintain conversational memory (last N turns) to support follow-up questions.

7.4 Backend (FastAPI)

- FR-13: `POST /chat` — accepts a query + session ID, returns answer + citations.
- FR-14: `POST /documents/upload` — accepts PDF files for ingestion.
- FR-15: `GET /documents` — lists indexed documents with metadata.
- FR-16: `DELETE /documents/{id}` — removes a document and its vectors from the index.
- FR-17: `GET /sessions/{id}/history` — retrieves conversation history.
- FR-18: `GET /health` — system health/status endpoint.

7.5 Evaluation

- FR-19: System shall run RAGAS evaluation (faithfulness, answer relevancy, context precision, context recall) on a curated Q&A test set.
- FR-20: System shall log evaluation results over time to detect regressions after pipeline changes.

7.6 Frontend / UI

- FR-21: Chat interface shall display streaming responses with inline, clickable citations.
- FR-22: Clicking a citation shall open a source viewer showing the exact page/passage highlighted.
- FR-23: Interface shall support multiple named conversation threads (sidebar history), like a professional workspace tool.
- FR-24: Interface shall include a document library view (browse/search indexed files).
- FR-25: Interface shall support light/dark themes matching enterprise branding.

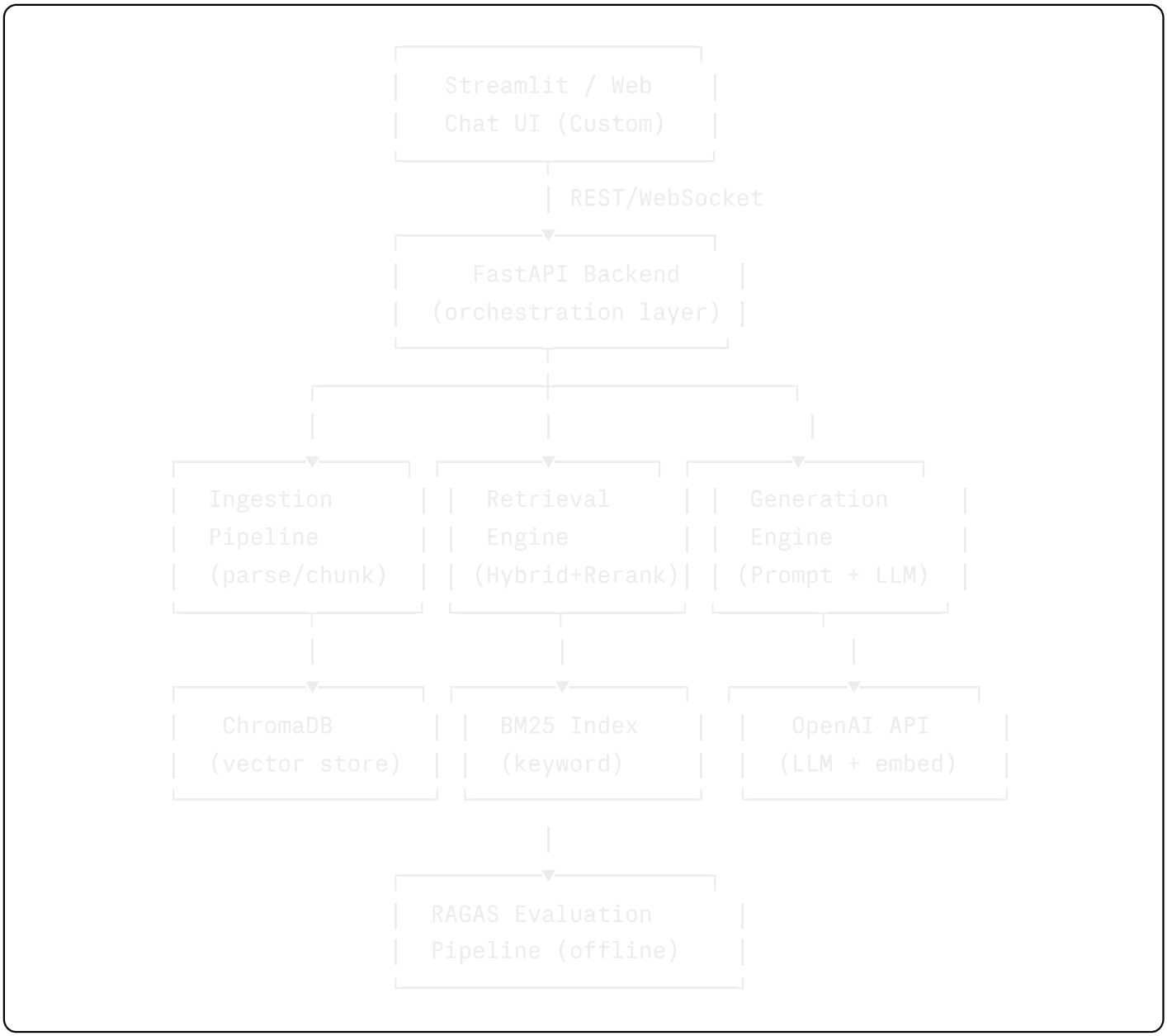
---

8. Non-Functional Requirements

Category	Requirement
Performance	P95 response latency ⩽ 4s for a query against a 500-page corpus
Scalability	Support 10,000+ pages indexed, 50+ concurrent users
Reliability	99.5% uptime for backend API
Security	Documents and vector store encrypted at rest; API authenticated (API key or SSO in Phase 2)
Data Privacy	No document content sent to third parties beyond the configured LLM provider; PII handling documented
Auditability	Every generated answer is traceable to exact source chunks used
Maintainability	Modular pipeline (ingestion / retrieval / generation are independently swappable)
Observability	Structured logging of queries, retrieval hits, latencies, and evaluation scores

---

9. System Architecture



10. Technology Stack

Layer	Technology
Backend API	FastAPI (Python)
Frontend	Streamlit (v1) → migration path to React/Next.js for production-grade custom UI
Vector Store	ChromaDB
Keyword Search	BM25 (rank-bm25 / Elasticsearch for scale)
Re-ranking	Cross-encoder (e.g., `ms-marco-MiniLM`)
LLM & Embeddings	OpenAI (GPT-4-class model + `text-embedding-3`)
Orchestration	LangChain
Evaluation	RAGAS
Deployment	Docker containers; cloud VM or Kubernetes

---

11. UI/UX Design Direction — "Doesn't Look Like AI"

A key product requirement is that the interface reads as a **serious internal enterprise tool** — closer to Notion, Linear, or a professional knowledge base — not a generic chatbot bubble UI with a robot avatar and purple gradients.

11.1 Design Principles

- **No AI clichés**: no robot/sparkle icons, no purple-to-blue gradients, no "Hi, I'm your AI assistant 🤖" copy.
- **Workspace metaphor, not chat-toy metaphor**: layout resembles a document workspace — sidebar with conversation threads and document library, main panel for Q&A, right panel for source preview.
- **Typography-first, content-dense**: clean serif/sans pairing, generous whitespace, but information-dense like an analyst tool (think Bloomberg Terminal meets Notion, not a mobile messaging app).
- **Citations as first-class UI elements**: inline citation chips (e.g., `[HR-Policy-2026, p. 12]`) are clickable pills, not footnotes — clicking opens a side panel with the highlighted source passage.
- **Neutral, confident color palette**: deep charcoal / off-white base, one restrained accent color (e.g., navy or forest green) — no neon, no default "AI purple."

- **Human tone in copy**: system messages read like a knowledgeable colleague ("No mention of this in the indexed documents — try rephrasing or check with HR directly") rather than assistant-speak.
- **Static, sturdy layout**: minimal animation; only subtle transitions (fade-in for streaming text, no bouncing avatars).

11.2 Key Screens

1. **Workspace Home** — document library grid with upload status, last-indexed date, page counts.
2. **Q&A Thread View** — left: thread list; center: conversation with inline citations; right (collapsible): source viewer.
3. **Source Viewer Panel** — renders the actual PDF page with the cited passage highlighted.
4. **Admin/Quality Dashboard** — RAGAS scores over time, ingestion status, query volume trends.

11.3 Interaction Details

- Streaming answers token-by-token, with citation chips appearing as they're referenced (not all at the end).
- Keyboard-first: `Cmd/Ctrl + K` to search documents, ↑ to edit last query.
- Empty/error states are informative, not cute — e.g., a plain message with a suggested next action, no mascot illustrations.

---

12. Evaluation Plan (RAGAS)

Metric	What it Measures	Target
Faithfulness	Does the answer avoid claims unsupported by retrieved context?	⩾ 0.85
Answer Relevancy	Is the answer actually addressing the question asked?	⩾ 0.85
Context Precision	Are the retrieved chunks relevant (low noise)?	⩾ 0.80
Context Recall	Did retrieval surface all the info needed to answer fully?	⩾ 0.80

Process: Maintain a curated golden test set (50–100 Q&A pairs per document domain) reviewed by subject-matter experts. Run RAGAS after every pipeline change (chunking strategy, re-ranker swap, prompt update) and block deployment if scores regress beyond a defined threshold (e.g., >5% drop).

---

13. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
LLM hallucination despite grounding	Medium	High	Strict prompt constraints + "insufficient information" fallback + faithfulness monitoring
Poor chunking splits context awkwardly	Medium	Medium	Use overlap + semantic chunking; evaluate context recall regularly
Latency grows with corpus size	Medium	Medium	Index sharding, caching frequent queries, async retrieval
Sensitive document leakage via LLM API	Low	High	Data processing agreement with LLM provider; consider self-hosted embeddings for sensitive corpora
Low adoption due to "just another chatbot" perception	Medium	High	Invest in the differentiated, non-generic UI (Section 11) and clear ROI messaging

---

14. Milestones & Rollout Plan

Phase	Milestone	Duration
Phase 0	Requirements finalization, corpus selection, golden test set creation	1 week
Phase 1	Ingestion pipeline (parsing, chunking, indexing)	2 weeks
Phase 2	Hybrid retrieval + re-ranking implementation	2 weeks
Phase 3	Generation pipeline + FastAPI backend	2 weeks
Phase 4	Custom UI build (workspace-style, non-generic)	3 weeks
Phase 5	RAGAS evaluation integration + tuning loop	1–2 weeks
Phase 6	Internal pilot with one department	2 weeks
Phase 7	Feedback incorporation + GA rollout	1–2 weeks

Total estimated timeline: ~13–15 weeks

---

15. Open Questions

1. Which document set(s) launch first — HR policies, legal contracts, or engineering docs?
2. Self-hosted embedding/LLM required for compliance, or is OpenAI API acceptable given data policies?
3. Do we need per-user access control on specific document sets at launch, or is that strictly Phase 2?
4. What's the retention policy for conversation history/session logs?

---

16. Appendix

- **RAGAS**: https://github.com/explodinggradients/ragas
- **LangChain**: https://python.langchain.com
- **ChromaDB**: https://www.trychroma.com